README

ota-android-agent

Field	Value
Revision	2
Created	2026-06-07
Status	implemented (`:core` fully unit-tested; `:android` real, AGP — see `BUILD_STATUS.md`)
Part of	Helix OTA
Gradle modules	`:core` (Kotlin/JVM), `:android` (Android library)
Language	kotlin (Kotlin 2.x, JVM toolchain 17; AGP 8.5.2, WorkManager)
License	Apache-2.0

Overview

`ota-android-agent` is the on-device OTA agent: it polls the control plane (interval + jitter), downloads an assigned update, **verifies it before apply**, hands the verified package to the OS apply path, and reports telemetry. The repo splits into two Gradle modules (`settings.gradle.kts`):

- **`:core`** — a **PURE Kotlin/JVM** library (no Android plugin, zero external runtime deps) holding all framework-independent logic: protocol DTOs + a dependency-free JSON codec, the verify-before-apply decision, the poll state machine, and jitter. Fully unit-tested on the JVM.
- **`:android`** — ONLY the Android-framework wiring: a `WorkManager` worker, the poll scheduler, and the `ApplyPort` (driving `update_engine` via reflection). It depends on `:core` for all logic.

Decoupling (§11.4.28): the agent does **not** hard-depend on the `ota-update-engine-bridge` artifact — it depends only on a small `ApplyPort` interface; the verify gate guarantees a poisoned/wrong artifact **never reaches update_engine** (`VerifyBeforeApply.kt`, `OtaPollWorker.kt`).

Public API

`:core`

- **Protocol** (`core/.../protocol/`)
  - DTOs (`Dtos.kt`): `PayloadProperties` (`asHeaderArray()`), `UpdateCheckRequest`, `UpdateAvailable`, `DeviceHealth`, `TelemetryEventRecord`, `TelemetryReport`, `TelemetryAck`.
  - Enums (`Enums.kt`): `UpdateState` (7 values incl. `IDLE`) and `TelemetryEvent` (6 values, no `idle`); both have `wire` + `companion.fromWire(String)`.
  - `OtaCodec` (`Codecs.kt`): `toJson(...)` / `updateAvailable(json)` / `updateCheckRequest(json)` / `telemetryReport(json)` / `telemetryAck(json)` / `payloadProperties(obj)` / `encode(JsonValue)` — wire field names match the REST contract.
- **JSON** (`core/.../json/Json.kt`): a minimal dependency-free `JsonValue` model + `Json.parse/write/obj/arr/str/num/bool` and typed accessors (`asObj`, `str`, `long`, `obj`, `arr`, `…OrNull`).
- **Verify-before-apply** (`core/.../verify/VerifyBeforeApply.kt`): `VerifyBeforeApply.decide(actualSha256, expectedSha256, signatureValid): Decision` (`Decision.Apply` / `Decision.Reject(RejectReason)`); ordered gates `MALFORMED_DIGEST → HASH_MISMATCH → SIGNATURE_INVALID`; case-insensitive, trimmed hash compare.
- **Poll state machine** (`core/.../poll/PollStateMachine.kt`): pure transitions `onPoll`, `onDownload`, `onVerify`, `onApplyStart`, `onApplyResult` over `PollState` (terminal `Success`/`Retry`/`Failed(reason)`); inputs `PollOutcome`, `DownloadOutcome`. Illegal transitions throw (the verify gate cannot be skipped).
- **Jitter** (`core/.../poll/Jitter.kt`): `Jitter.nextDelayMillis(baseMillis, jitterMaxMillis, rng): Long` — `base + uniform[0, jitterMax)`, with an **injectable** RNG for deterministic tests.

`:android` (`package digital.vasic.helix.ota.agent`)

- `ApplyPort` interface + `VerifiedPackage` + `ApplyResult` (`apply/ApplyPort.kt`) — the decoupling boundary; `applyVerified(pkg): ApplyResult`.
- `ReflectiveUpdateEngineApplyPort` (`apply/ReflectiveUpdateEngineApplyPort.kt`) — drives `android.os.UpdateEngine.applyPayload(file://…, offset, size, headers)` via reflection; returns `Failed` (never fabricates success) off a system build.
- `OtaPollWorker` (`poll/OtaPollWorker.kt`) — `CoroutineWorker` running one poll→download→verify→apply cycle; `companion.runCycle(deps)` is the JVM-testable driver; `AgentDependencies` injects the ports.
- `PollScheduler` + `PollConfig` (`poll/PollScheduler.kt`) — `schedule(wm, cfg, rng)`; 15-min periodic work with jitter and exponential backoff.

- Ports (`poll/Ports.kt`): `ControlPlaneClient`, `Downloader` (+ `DownloadResult`), `Verifier` (+ `VerifyResult`), `Telemetry`, and `PollResult` (+ `toOutcome()`).

Usage

```
// Pure verify-before-apply gate (:core) — runs on any JVM, no
        Android needed:
import digital.vasic.helix.ota.core.verify.VerifyBeforeApply
import digital.vasic.helix.ota.core.verify.Decision

val decision = VerifyBeforeApply.decide(
    actualSha256   = "9F86D0...",   // from Security-KMP over the
        downloaded bytes
    expectedSha256 = "9f86d0...",   // from the control-plane
        manifest
    signatureValid = true,
)
if (decision is Decision.Apply) { /* safe to hand the local file
        to update_engine */ }

// Schedule the periodic poll worker (:android):
import digital.vasic.helix.ota.agent.poll.PollScheduler
import digital.vasic.helix.ota.agent.poll.PollConfig

PollScheduler.schedule(WorkManager.getInstance(context),
        PollConfig(periodMinutes = 15))
```

Testing

The `:core` module is the fully-tested deliverable; run its JVM unit tests:

```
cd submodules/ota-android-agent
gradle --no-daemon --console=plain :core:test
```

`:core` tests (`core/src/test/.../core/`) cover: the poll state machine happy path and every branch — no-update/transient/download-failure/apply-throw, the **verify-reject hard-failure that never reaches apply**, and illegal-transition guards (`PollStateMachineTest.kt`); jitter bounds, seeded determinism, zero/zero, and negative-arg rejection (`JitterTest.kt`); DTO ⇄ JSON round-trips for `UpdateAvailable`, `UpdateCheckRequest`, `TelemetryReport` (all six event types, null health/deployment), `TelemetryAck`, the exact enum value sets, unknown-wire-value rejection, JSON escaping, and the `asHeaderArray` applyPayload contract (`CodecRoundTripTest.kt`); and the verify-before-apply decision incl. ordering precedence and a mutation-immunity check that an inverted hash compare would flip the decision (`VerifyBeforeApplyTest.kt`).

`:android` is real Kotlin (no stubs), but its build runs the Android Gradle Plugin under the system Gradle — see `BUILD_STATUS.md` for the honest per-module build/test record in this environment.

Reusable building brick

This is a **reusable, independently versioned** Helix OTA building brick (HelixConstitution §11.4.28 — submodules-as-equal-codebase). It consumes the OTA contracts (`ota-protocol` shapes, mirrored in `:core`) and reaches the OS apply path through the decoupled `ApplyPort` (conceptually backed by `ota-update-engine-bridge`), with no server logic. Universal constitution rules are inherited via this repo's `CLAUDE.md` / `AGENTS.md` (`## INHERITED FROM Helix Constitution`).

Mirrors

- GitHub: https://github.com/HelixDevelopment/ota-android-agent
- GitLab: https://gitlab.com/helixdevelopment1/ota-android-agent